fractal

Session 5:

Ship Your PoC

Certificate in AI-Assisted Development | Session 4 | ~2 Hours | Claude Code | Final Session

Session 3 Recap

What You Produced

DEBUGGED

- ✓ 5-step debugging loop applied to a broken project
- ✓ Root cause named before every fix
- ✓ Regression test written and passing
- ✓ Habit established: read the diff before accepting

REVIEWED

- ✓ Structured review run on your own PoC
- ✓ Issues classified: Critical, High, Medium, Low
- ✓ Critical and High issues fixed and re-verified
- ✓ Medium issues noted in README

COMMITTED & DOCUMENTED

- ✓ Reviewed branch created with a descriptive commit message
- ✓ HANDOFF.md generated, edited, and committed to Git

You have a working, reviewed, documented PoC. Today you find out what it would actually take to ship it.

fractal

Learning Outcomes

By the end of this session, you will be able to:

01 Apply the production gap checklist and 10x rule to estimate the engineering effort required to take your PoC to production

02 Generate and critique a data model for your PoC domain using Claude Code to surface structural assumptions

03 Identify data privacy, security, and governance risks in your PoC and produce a brief risk assessment

01 PoC To Production

Why Most PoCs Never Ship

The problem is rarely the PoC itself. It is the gap between what a PoC proves and what production requires.

THE DEMO PROBLEM

- Built for one question, one dataset, one predictable input.
- Production has none of these.

THE COMPLEXITY PROBLEM

- Auth, error recovery, audit logging, monitoring, rate limiting — none of it was forgotten. It was deliberately left out.
- Production puts it all back.

THE EXPECTATION PROBLEM

- Stakeholder saw a working demo.
- Engineering sees 10–20% of what production needs.
- Both are right. This session bridges that gap.

A PoC that never ships is not a failure. A PoC whose production cost was never estimated is a missed conversation.

The Production Gap

What Claude Code builds vs. what production needs. Production requires everything you forgot to ask for.

WHAT YOUR PoC HAS	WHAT PRODUCTION NEEDS
Core logic that runs	Core logic that runs at scale, with concurrent users
Hardcoded credentials	Secrets management and credential rotation
CLI output	Logging, monitoring, and alerting
One tested path	Full error handling and graceful failure
A README	Architecture docs, API contracts, runbooks
Committed to Git	CI/CD pipeline and automated tests
One user, you	Authentication and authorisation
Works on your machine	Works on any machine, any time, any load

None of the right column is a criticism of the PoC. All of it is the honest cost of production.

Technical Gaps from AI Code

In general, AI-generated code has a predictable pattern of gaps. Knowing them lets you find them before engineering does.

No input validation	Hardcoded secrets	No retry logic	Brittle dependencies
• Built for the input you described, not the one you forgot • Every unvalidated input is a crash waiting to happen	• API keys and passwords written directly into the code • Fine in a PoC. A security incident in production.	• External API fails, the script crashes • Claude will not add retry or graceful failure unless asked	• Library versions from Claude's training data • May not be current, maintained, or org-approved

These are not mistakes. They are the natural output of code built for proof, not production. To fix this, we have 10x rule.

fractal

The 10x Rule

A practical framework for estimating the real engineering effort to productionise a PoC, and for communicating it honestly to stakeholders.

THE RULE

For every unit of effort it took to build the PoC, estimate 10 units to make it production-ready.

A PoC built in one session means roughly 10 sessions of engineering effort to ship it.

This is a heuristic, not a formula. The multiplier shrinks for simple scripts with no external dependencies, and grows for anything touching authentication, compliance, or scale.

WHY IT EXISTS

The 10x rule is not pessimism, it is calibration. The gap is structurally large because:

- Security, compliance, and observability were all out of scope by design
- AI-generated code assumes ideal conditions; production provides none
- Code that works once needs to work every time, for every user, when things go wrong

When a stakeholder asks, "how hard to build can it be?", the 10x rule is your answer.

fractal

Ask Before Going to Production?

Now that you can see the full gap and the spec it would take to close it, ask these four questions in order.

01

Does it answer a real question with measurable value?

If you cannot name the business metric that improves, the PoC should not go to production, no matter how well it runs.

02

Is the production gap closable within a realistic budget and timeline?

Apply the 10x rule. If the estimate exceeds what the business will invest, the conversation ends here, not in month four.

03

Does engineering have what they need to pick it up?

CLAUDE.md, HANDOFF.md, review findings, a clear scope boundary. If any of these are missing, it is not ready to hand over.

04

Is there a named owner on the other side?

Production code needs someone to maintain it, monitor it, and fix it. No named owner means no production path.

Two yeses and two nos: rework. Four yeses: greenlight. Any no on question 1 or 2: kill it now and learn from it.

fractal

02 ONTOLOGY & STRUCTURAL THINKING

What Ontology Means

The vocabulary, entities, and relationships that define how a system understands its domain.

THE PLAIN ENGLISH VERSION

Ontology answers three questions about your system:

- What things exist? Entities, like accounts, deals, contacts, reports.
- How do they relate? Relationships, like an account has many contacts.
- What do we call them? Vocabulary: is it a “deal” or an “opportunity”? The name matters more than it looks.

Why You Should Care About Ontology?

The upstream decisions that constrain everything downstream, made at the PoC stage whether you are paying attention or not.

SCHEMA DESIGN SHAPES EVERYTHING

- Determines what questions you can ask the data
- How fast you can query it, how hard it is to change
- These decisions get made in the first few prompts

NAMING CONVENTIONS TRAVEL

- Whatever Claude named your entities appears everywhere
- Database, API, frontend, docs
- Changing them after production is a migration, not a rename

MISSING RELATIONSHIPS CREATE DEBT

- PoCs model one entity cleanly, leave relationships implicit
- When engineering adds a second entity, that implicit relationship becomes an expensive migration

You do not need to design the data model. You need to review it before engineering inherits it.

Common Ontology Mistakes

Claude models what you described. These are the structural assumptions it makes that you may not have intended.

✗ Implicit assumptions in variable names	→	✓ Ask: “What assumptions did you make in naming the fields of this data model?”
✗ Fields that conflate two concepts	→	✓ Ask: “Are there any fields that carry more than one meaning? Name them.”
✗ Missing relationships between entities	→	✓ Ask: “What relationships are implied but not modelled explicitly?”
✗ IDs that will not survive at scale	→	✓ Ask: “What would need to change to support multiple users or organisations?”

fractal

Generate & Review a Data Model

Two prompts. One to make the model visible. One to find what it got wrong.

GENERATE

```
claude "Generate an entity-
relationship diagram for this
project in plain text. List
every entity, every field, and
every relationship."
```

What you get: a structured map of the data model Claude built, often with assumptions you did not know had been made.

REVIEW

```
claude "Review this data model
critically. What assumptions
did you make? What breaks
with more than one user?
What is missing?"
```

What you get: a list of structural risks, named specifically, before engineering inherits them.

The generate prompt makes the implicit explicit. The review prompt makes the risky visible. Run both before you hand over.

Ontology Sprint

HANDS-ON LAB

Generate the data model Claude built for your PoC. Critique it. Find the assumption you did not know was there.

1 **Generate the entity-relationship model**

```
claude "Generate an entity-relationship diagram for this project
in plain text. List every entity, field, and relationship."
```

2 **Read the model before critiquing it**

Does it match what you intended? Are the names right? Are the relationships complete?

3 **Critique the model**

```
claude "Review this data model critically. What assumptions didyou make? What breaks at scale? What is missing?"
```

4 **Name your one structural insight**

The assumption you made that you did not know you were making. One sentence.

5 **Make one targeted fix**

Pick the most significant issue. Ask Claude to show the corrected model.

✓ **Model generated · Model read and understood · Critique run · Structural insight named · Targeted fix applied**

fractal

Thank You.

fractal

A strategic partner to the most admired Fortune 500® companies globally, we help power every human decision in the enterprise by bringing advanced analytics & AI, engineering and design.

www.fractal.ai